

UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Buenos Aires Carrera:

Tecnicatura Universitaria en Programación

Materia: Programación I

Profesor: Martín Alejandro García

TRABAJO PRÁCTICO INTEGRADOR

Tema: Gestión de Datos de Países en Python: filtros,
ordenamientos y estadísticas

Estudiante: Enzo Luciano Jalil

Fecha de Entrega: Junio de 2026

Título del tema

Desarrollo de un sistema modular de consola en Python para la administración,
consistencia y análisis estadístico de un dataset de países con persistencia de
datos en formato CSV.

ÍNDICE

1. Marco teórico	Pág 3
I. Listas	Pág 3
II. Diccionarios	Pág 4
III. Funciones	Pág 5
IV. Condicionales	Pág 6
V. Ordenamientos	Pág 8
VI. Estadísticas básicas	Pág 10
VII. Archivos CSV	Pág 11
2. Decisiones técnicas y arquitectura	Pág 13
I. Diagrama del proyecto	Pág 13
II. Justificación de la arquitectura y estructura de datos	Pág 13
III. Capturas de pantalla en funcionamiento	Pág 14
• Menú principal	Pág 14
• Proceso de carga exitoso	Pág 15
• Proceso de carga fallido	Pág 15
• Resultado de filtrado	Pág 16
• Reporte de estadísticas	Pág 17
3. Dificultades y conclusiones	Pág 17
• Sobre la modalidad de trabajo	Pág 17
• Obstáculos técnicos y cómo los resolví	Pág 17
• Conclusión	Pág 18
4. Bibliografía / Webgrafía	Pág 18
5. Links	Pág 19

1. Marco teórico

I. Listas:

- **Definición:** Una lista es una secuencia de valores, donde dichos valores (llamados elementos o ítems) pueden ser de cualquier tipo.
- **Sintaxis:** Se crean encerrando los elementos entre corchetes [] y separándolos por comas.
- **Mutabilidad:** A diferencia de las cadenas de texto, las listas son mutables, lo que significa que sus elementos pueden ser modificados, reasignados o eliminados después de su creación.
- **Indexación:** Los elementos se acceden mediante índices enteros que comienzan en 0.
- **Anidamiento:** Una lista puede contener otras listas como elementos, lo cual es útil para representar estructuras de datos complejas.
- **Procesamiento (Mapa, Filtro y Reducción):** Son patrones para manipular colecciones: el mapa transforma cada elemento, el filtro selecciona una sublista bajo un criterio y la reducción combina los elementos en un único resultado (como una suma total).
- **Ejemplos:**

#Crear una lista de cosas y buscar el primer elemento
Colores = ["rojo", "azul", "verde"]
primer_color = colores[0] # Da "rojo"
Cambiar un dato directamente por su posición
colores[1] = "amarillo" # Ahora la lista es ["rojo", "amarillo", "verde"]
Filtro conceptual: quedarme solo con números grandes
valores = [5, 20, 8, 35]
filtrados = [x for x in valores if x > 10] # Da [20, 35]

- **Bibliografía:**
Downey, A. (2015). Think Python: Aprende a pensar como un informático (2da ed.). (J. Espinoza, Trad.). Green Tea Press.

II. Diccionarios:

- **Definición:** Un diccionario es una estructura de datos que representa un **mapeo** entre una colección de índices (llamados **claves**) y una colección de **valores**.
- **Asociación:** A diferencia de las listas, donde los índices deben ser enteros, en un diccionario las claves pueden ser de casi cualquier tipo, siempre que sean **hashables** (inmutables).
- **Implementación:** Se basan en una estructura interna denominada **tabla hash**, lo que permite que operaciones como la búsqueda o la verificación de pertenencia (operador `in`) se realicen en **tiempo constante**, independientemente de la cantidad de elementos.
- **Mutabilidad:** Los diccionarios son objetos **mutables**, permitiendo agregar, eliminar o modificar pares clave-valor después de su creación.
- **Orden:** Los elementos en un diccionario no mantienen un orden predecible ni específico; se accede a la información directamente a través de sus claves y no por una posición numérica secuencial.
- **Ejemplos:**

1. Creación (Claves inmutables y valores de cualquier tipo)
<pre> dicc = { "nombre": "Ana", 42: "Suerte", (1, 2): "Coordenadas" }</pre>
2. Mutabilidad (Agregar, modificar y eliminar)
<pre> dicc["edad"] = 28 # Agregar dicc["nombre"] = "Eva" # Modificar del dicc[42] # Eliminar</pre>
3. Acceso directo e Implementación (Búsqueda rápida por clave, no por posición)
<pre>print(dicc["nombre"]) # Imprime: Eva</pre>
Operador 'in' (Verificación en tiempo constante gracias a la tabla hash)
<pre> if "edad" in dicc: print(dicc["edad"]) # Imprime: 28</pre>

- **Bibliografía:**

Downey, A. (2015). Think Python: Aprende a pensar como un informático (2da ed.). (J. Espinoza, Trad.). Green Tea Press.

III. Funciones:

- **Definición:** Una función es una secuencia de instrucciones con nombre que realiza una tarea específica.
- **Modularización:** Su propósito principal es dividir un programa complejo en partes más pequeñas y manejables, facilitando la lectura, depuración y reutilización del código.
- **Estructura de definición:** Se utiliza la palabra clave def, seguida del nombre de la función y paréntesis que pueden contener parámetros.
- **Parámetros y Argumentos:** Los parámetros son variables internas que reciben la información, mientras que los argumentos son los valores reales que se le pasan a la función al llamarla.
- **Ámbito local:** Las variables y parámetros definidos dentro de una función son locales, lo que significa que solo existen mientras la función se ejecuta y no interfieren con el resto del programa.
- **Sentencia return:** Permite que una función termine su ejecución y devuelva un resultado específico al punto donde fue llamada.
- **Clasificación:**
 - a. **Funciones productivas:** Aquellas que calculan y devuelven un valor de retorno útil.
 - b. **Funciones nulas:** Aquellas que realizan una acción (como imprimir en consola) pero devuelven técnicamente el valor None.
- **Ejemplos:**

<pre># --- 1. Definición, Estructura y Modularización --- def calcular_iva(precio): # Parámetro y palabra clave 'def' # --- Ámbito Local --- impuesto = precio * 0.21 # 'impuesto' solo existe dentro de la función # --- Funciones Productivas (Sentencia return) --- return precio + impuesto # Termina y devuelve un valor útil</pre>
<pre># --- 2. Parámetros vs Argumentos y Reutilización --- total = calcular_iva(100) # 100 es el argumento real. 'total' guarda el retorno. # --- 3. Funciones Nulas (Devuelven None) --- def mostrar_total(): print(f"Total: {total}") # Realiza una acción (imprimir) pero no retorna nada retorno = mostrar_total() # Ejecuta la acción print(retorno) # Imprime: None (demuestra que es una función nula)</pre>

- **Bibliografía**

Downey, A. (2015). Think Python: Aprende a pensar como un informático (2da ed.). (J. Espinoza, Trad.). Green Tea Press.

IV. Condicionales:

- **Definición:** Los condicionales le dan al programa la capacidad de decidir qué camino seguir o qué código ejecutar dependiendo de si se cumple o no una condición.
- **Expresiones booleanas:** Son expresiones que solo pueden tener dos resultados posibles: verdadero (True) o falso (False).
- **Operadores de comparación:** Se usan para armar las condiciones; comparan valores para ver si son iguales (==), distintos (!=), mayores (>) o menores (<), entre otros.
- **Operadores lógicos:** Permiten combinar varias condiciones. Se usa and para que se cumplan todas, or para que se cumpla al menos una, y not para negar una condición.
- **Sentencia if:** Es la estructura básica donde el código interno solo se activa si la condición es verdadera.
- **Sentencia else:** Sirve para crear un camino alternativo; es lo que hace el programa si la condición del if resulta ser falsa.
- **Sentencia elif:** Es una forma de encadenar muchas opciones. El programa revisa cada una en orden y se detiene en la primera que sea verdadera.
- **Anidamiento:** Se refiere a poner un condicional dentro de otro para realizar chequeos más complejos, aunque se recomienda no abusar de ellos para que el código sea fácil de leer.
- **Sentencia pass:** Es un marcador que no hace nada; se usa para rellenar espacios donde la sintaxis de Python pide código pero todavía no queremos poner ninguna acción.

- **Ejemplos:**

<pre> clima = "lluvia" edad = 20 </pre>
Estructura Completa (if, elif, else, pass) y Anidamiento
<pre> if clima == "lluvia" and not edad > 18: pass # 'pass': marcador que no hace nada </pre>
<pre> elif clima != "sol" or edad >= 18: # encadena opciones: al menos una # --- Anidamiento (Un condicional dentro de otro) --- if edad == 20: print("Acceso permitido.") # Se ejecuta si la condición es True </pre>
<pre> else: print("Camino alternativo.") # 'else': se ejecuta si todo lo anterior es False </pre>

- **Bibliografía:**

Downey, A. (2015). *Think Python: Aprende a pensar como un informático* (2da ed.). (J. Espinoza, Trad.). Green Tea Press.

V. Ordenamientos:

- **Definición:** El ordenamiento es el proceso de organizar los elementos de una colección (como una lista) siguiendo un criterio específico, comúnmente de menor a mayor.
- **Método sort:** Es un método propio de las listas en Python que organiza sus elementos de forma interna (in-place). Es un método nulo, lo que significa que modifica la lista original y devuelve None.
- **Función sorted:** A diferencia de sort, esta función incorporada toma cualquier colección y devuelve una nueva lista ordenada, dejando la colección original sin cambios.
- **Criterios de ordenamiento:**
 - a. **Cadenas:** Se ordenan alfabéticamente, pero Python prioriza las letras mayúsculas sobre las minúsculas.
 - b. **Números:** Se ordenan por su valor numérico.
 - c. **Objetos personalizados:** Para ordenar objetos creados por el programador (como una lista de países), se debe definir el método especial `__lt__` ("menor que"), que le indica a Python qué atributo comparar (por ejemplo, el nombre o la población).
- **Rendimiento:** El algoritmo de ordenamiento de Python es muy eficiente; técnicamente tiene un orden de crecimiento de $O(n \log n)$, lo que lo hace adecuado para manejar grandes cantidades de datos.

- **Ejemplos:**

1. Método sort() (Modifica in-place, es método nulo y retorna None) nums = [5, 2, 9] retorno = nums.sort() print(nums, retorno) # [2, 5, 9] None
2. Función sorted() (Crea NUEVA lista; Criterio strings: Mayúsculas primero) palabras = ["gato", "Ana"] print(sorted(palabras)) # ['Ana', 'gato'] (Eficiencia: $O(n \log n)$)
3. Objetos personalizados (Uso de __lt__ para definir el criterio) class Pais: def __init__(self, nom, pob): self.nom = nom self.pob = pob def __lt__(self, otro): return self.pob < otro.pob # Criterio: menor a mayor población paises = [Pais("Bra", 214), Pais("Chi", 19)] paises.sort() print([p.nom for p in paises]) # ['Chi', 'Bra']

- **Bibliografía**

Downey, A. (2015). *Think Python: Aprende a pensar como un informático* (2da ed.). (J. Espinoza, Trad.). Green Tea Press.

VI. Estadísticas básicas:

- **Definición:** En el contexto de la programación, las estadísticas consisten en procesar colecciones de datos para obtener indicadores como totales, promedios o valores extremos.
- **Reducción (Reduction):** Es un patrón de procesamiento que recorre una secuencia de elementos y los combina para obtener un único valor de resultado. Se aplica, por ejemplo, al usar una función para sumar todas las poblaciones y calcular un promedio.
- **Acumulador:** Es una variable que se utiliza dentro de un bucle para ir sumando o acumulando un resultado paso a paso. Es la herramienta básica para obtener totales.
- **Contador:** Se refiere a una variable que se inicializa en cero y se incrementa (normalmente de a uno) cada vez que se encuentra un dato de interés. Es muy útil para tareas como contar cuántos países pertenecen a un continente específico.
- **Histograma:** Es un término estadístico que describe una colección de contadores o frecuencias. En Python, se implementa frecuentemente con diccionarios para registrar cuántas veces aparece cada categoría en un dataset.
- **Valores extremos:** Para encontrar los puntos máximos y mínimos (como el país con mayor o menor población), se utilizan algoritmos de búsqueda o funciones incorporadas como `max()` y `min()`, que identifican el valor más alto o bajo en una secuencia
- **Ejemplos:**

1. Acumulador, Contador y Reducción (Combinar datos en un único resultado) <pre>poblaciones = [10, 20, 30] total = 0 # Acumulador: inicia en cero cantidad = 0 # Contador: cuenta elementos for pob in poblaciones: total += pob # Acumula paso a paso cantidad += 1 # Incrementa de a uno promedio = total / cantidad # Reducción final: un único valor (20.0)</pre>
2. Histograma (Colección de contadores usando un diccionario) <pre>categorias = ["Arg", "Bra", "Arg"] histograma = {} for cat in categorias: # Si la clave existe, suma 1; si no, la crea en 0 y suma 1 histograma[cat] = histograma.get(cat, 0) + 1 # {'Arg': 2, 'Bra': 1}</pre>
3. Valores extremos (Búsqueda de máximos y mínimos) <pre>maximo = max(poblaciones) # 30 minimo = min(poblaciones) # 10</pre>

- **Bibliografía**

Downey, A. (2015). *Think Python: Aprende a pensar como un informático* (2da ed.). (J. Espinoza, Trad.). Green Tea Press.

VII. Archivos CSV

- **Persistencia:** Es la capacidad de los programas para guardar información en un medio de almacenamiento permanente (como el disco duro) de modo que los datos no se pierdan al cerrar el programa.
- **Definición de CSV:** Es un formato de archivo de texto (Valores Separados por Comas) donde cada línea representa un registro de datos y los campos individuales dentro de ese registro están divididos por un delimitador, comúnmente una coma.
- **Apertura con open:** Función fundamental para crear un enlace entre el programa y el archivo físico. Devuelve un "objeto de archivo" que permite realizar operaciones de lectura o escritura.
- **Lectura iterativa:** Los archivos pueden recorrerse línea por línea utilizando un bucle for, lo cual es eficiente para procesar grandes conjuntos de datos, como los registros de países.
- **Limpieza con strip:** Método de cadenas esencial para eliminar caracteres "invisibles" (como el salto de línea \n) que se encuentran al final de cada registro leído del archivo.
- **Procesamiento con split:** Esta función divide una línea de texto en una lista de elementos individuales basándose en el delimitador especificado, permitiendo separar, por ejemplo, el nombre de un país de su población.
- **Conversión de tipos:** Dado que los archivos de texto almacenan toda la información como caracteres (str), los campos numéricos deben convertirse manualmente (usando int() o float()) para poder realizar operaciones estadísticas con ellos.
- **Finalización con close:** Es la instrucción que cierra el archivo al terminar de usarlo, asegurando que se liberen los recursos del sistema correctamente

- **Ejemplos:**

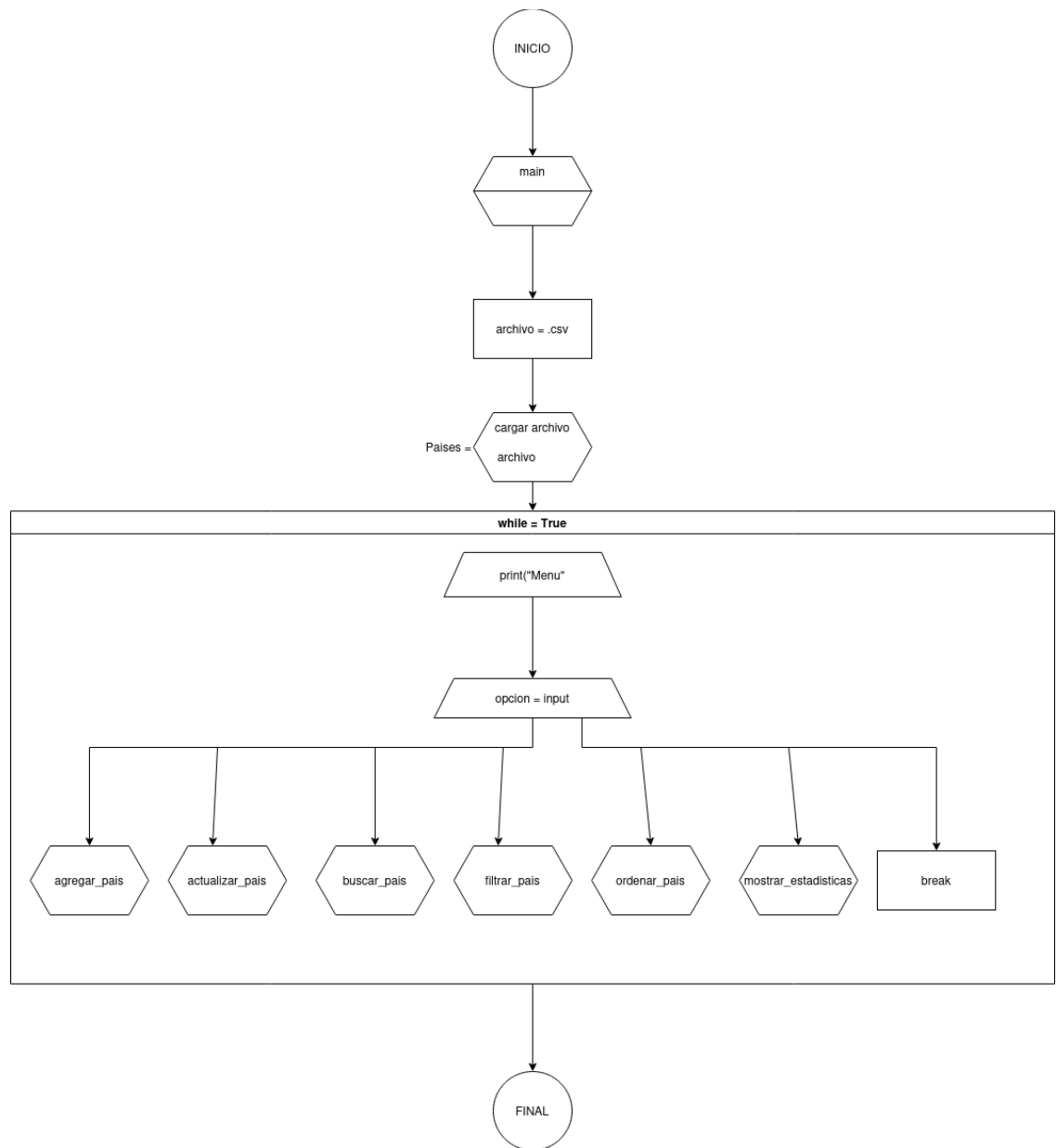
```
#1. Apertura, Persistencia y Lectura Iterativa (Línea por línea)'open'  
enlaza el archivo en disco permanente para leer sus datos  
  
archivo = open("datos.csv", "r")  
for linea in archivo: # Recorre de forma eficiente sin cargar todo a la  
memoria  
# 2. Limpieza (strip), Procesamiento (split) y Conversión de tipos  
# datos.csv contiene líneas como: "Argentina,45000000\n"  
linea_limpia = linea.strip() # Elimina el salto de línea '\n'  
campos = linea_limpia.split(",") # Divide por delimitador: ['Argentina',  
'45000000']  
  
nombre_pais = campos[0]  
poblacion = int(campos[1]) # Conversión manual: de str a int para  
operar  
  
print(f" {nombre_pais}: {poblacion}")  
  
#3. Finalización  
archivo.close() # Cierra el archivo y libera los recursos del sistema
```

- **Bibliografía**

Downey, A. (2015). *Think Python: Aprende a pensar como un informático* (2da ed.). (J. Espinoza, Trad.). Green Tea Press.

2. Decisiones técnicas y arquitectura:

I. DIAGRAMA DEL PROYECTO:



II. Justificación de la arquitectura y estructura de datos:

Para el desarrollo de este sistema, se tomaron las siguientes decisiones técnicas basadas en los requerimientos del proyecto:

- Estructura de Datos Principal: Se optó por utilizar una lista de diccionarios para representar el dataset de países en la memoria RAM. Esta elección permite combinar la mutabilidad de las listas para gestionar el orden de los registros con la claridad de los diccionarios, que asocian cada atributo del país (nombre, población, superficie) a una clave descriptiva.

- Modularización del Código: Siguiendo el principio de "una función = una responsabilidad", el sistema fue dividido en módulos independientes para separar la lógica de negocio (consultas y estadísticas) de la persistencia de datos (gestión de archivos CSV) y las validaciones de entrada.
- Estrategia de Persistencia: Se implementó el manejo de archivos en formato CSV para garantizar la persistencia de la información. El flujo de ejecución asegura que los datos se carguen íntegramente al inicio y se guarden de forma permanente en el disco al finalizar las operaciones.
- Validaciones y Control de Errores: Se incluyeron estructuras condicionales y bloques de manejo de excepciones para evitar fallos por ingresos de datos inválidos, campos vacíos o búsquedas sin resultados, garantizando una interacción fluida con el usuario

III. Capturas de pantalla en funcionamiento:

- Menú principal

```
daku@DakuPc:~$ cd /home/daku/Documentos/GitHub/Enzo_Luciano_Jalil_TPI/caso_practico/
python3 main.py
¡Datos cargados con éxito!

=====
      SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
1. Agregar país
2. Actualizar población/superficie
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción (1-7): █
```

- **Proceso de carga exitoso**

```
daku@DakuPc:~$ cd /home/daku/Documents/GitHub/Enzo_Luciano_Jalil_TPI/caso_practico/
python3 main.py
¡Datos cargados con éxito!

=====
      SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
1. Agregar país
2. Actualizar población/superficie
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción (1-7): 1

--- AGREGAR PAÍS ---
Ingrese el nombre del país: México
Ingrese la población (solo números enteros): 129000000
Ingrese la superficie en km² (solo números enteros): 1964375
Ingrese el continente: América
¡País 'México' agregado y guardado con éxito!

Presione ENTER para continuar...)
```

- **Proceso de carga fallido**

```
=====
      SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
1. Agregar país
2. Actualizar población/superficie
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción (1-7): 1

--- AGREGAR PAÍS ---
Ingrese el nombre del país: mexico
Error: El país 'México' ya se encuentra registrado.
Por favor, intente con otro nombre.

Ingrese el nombre del país: 1
ERROR: Ingrese solo letras.
Ingrese el nombre del país: 
```

- **Resultado de filtrado**

```
daku@DakuPc:~$ cd /home/daku/Documentos/GitHub/Enzo_Luciano_Jalil_TPI/caso_practico/
python3 main.py
¡Datos cargados con éxito!

=====
  SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
1. Agregar país
2. Actualizar población/superficie
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción (1-7): 4

--- FILTRAR PAÍSES ---
1. Por Continente
2. Por Rango de Población
3. Por Rango de Superficie
4. Volver al menú principal

Seleccione una opción de filtrado: 1
Ingrese el continente a buscar: europa

--- SE ENCONTRARON 4 COINCIDENCIAS ---
- Alemania | Continente: Europa | Población: 83149300 | Superficie: 357022 km²
- Francia | Continente: Europa | Población: 68300000 | Superficie: 543965 km²
- España | Continente: Europa | Población: 48500000 | Superficie: 505992 km²
- Italia | Continente: Europa | Población: 58900000 | Superficie: 301340 km²

Presione ENTER para continuar... 
```


- **Reporte de estadísticas**

```
=====
SISTEMA DE GESTIÓN DE PAÍSES (UTN)
=====
1. Agregar país
2. Actualizar población/superficie
3. Buscar país por nombre
4. Filtrar países
5. Ordenar países
6. Mostrar estadísticas
7. Salir
=====
Seleccione una opción (1-7): 6
=====
REPORTE ESTADÍSTICO GENERAL
=====
• Total de países registrados: 15
• País con MAYOR población: India (1,440,000,000 hab.)
• País con MENOR población: Australia (26,800,000 hab.)
• Promedio de población: 280,261,300.00 habitantes
• Promedio de superficie: 3,864,317.20 km²

--- Cantidad de países por continente ---
- América: 5 países
- Asia: 3 países
- Europa: 4 países
- África: 2 países
- Oceanía: 1 país
=====
Presione ENTER para continuar... 
```

3. Dificultades y conclusiones:

Sobre la modalidad de trabajo Por autorización del tutor, realicé este proyecto de forma individual. Debido a problemas del corte de servicios básicos en mi hogar, se me hacía muy difícil coordinar horarios y tareas con un compañero, así que preferí organizarme solo.

Obstáculos técnicos y cómo los resolví, aunque el desarrollo fue fluido gracias a que fui bastante organizado, tuve un par de desafíos técnicos:

- Acentos y búsquedas: Al principio, si buscaba un país como "Japón" sin tilde, el sistema no lo encontraba. Lo solucioné investigando cómo "limpiar" el texto con una función de normalización para que las búsquedas no fallen por un acento.
- Datos del CSV: Me di cuenta de que, al leer el archivo, los números (población y superficie) entraban como texto. Tuve que convertirlos a enteros manualmente para que las estadísticas y promedios no den error.

- Modularización: Separar el código en varios archivos fue un reto para que todo conecte bien, pero me ayudó a que cada parte del programa tenga una sola responsabilidad.
- Conclusión Este trabajo me sirvió para entender bien cómo se usan las listas y diccionarios en un caso real. Aprendí que ser ordenado con las funciones y separar el código por archivos hace que todo sea mucho más fácil de manejar y corregir. Fue un desafío personal por las complicaciones que tuve, pero logré cumplir con todos los requerimientos. Este proyecto me permitió consolidar el concepto de **abstracción** y entender la importancia de la persistencia de datos para que la información no se pierda al cerrar el programa.

4. Bibliografía / Webgrafía

Bibliografía Base:

- Downey, A. (2015). *Think Python: Aprende a pensar como un informático* (2da ed.). (J. Espinoza, Trad.). Green Tea Press.
- Documentación Oficial y Herramientas de Apoyo:
- Python Software Foundation (2025). *csv — CSV File Reading and Writing*. Documentación oficial de Python 3.
- Python Software Foundation (2025). *unicodedata — Unicode Database*. Documentación oficial de Python 3.
- Google AI (2024). *Gemini* [Modelo de lenguaje extenso]. Recuperado de: <https://gemini.google.com> (Utilizado como recurso de apoyo).

Recursos de la Cátedra:

- Universidad Tecnológica Nacional (UTN). (2025). *Consigna Trabajo Práctico Integrador (TPI): Gestión de Datos de Países en Python*. Tecnicatura Universitaria en Programación a Distancia (TUPAD).

5. Links:

- **Repositorio GitHub:**
https://github.com/dakucrow/Enzo_Luciano_Jalil_TPI
- **Link video explicativo:**
https://youtu.be/_ii1F_GkNuE?si=xo3oe2nVR2U8ia3n